GEMM Kernel Optimization Report — GEAK Kernel Workflow

1. Overview: GEAK and Kernel Workflow

GEAK is multi-agent GPU performance optimization for AMD Instinct MI GPUs (CDNA, e.g. gfx942 / gfx950 — the on-box card is auto-detected). Driven by Claude Code, orchestrated by deterministic JS Workflows.

`kernel_workflow/` optimizes a single AMD GPU kernel — Triton, HIP, CK, FlyDSL, or any AMD GPU source:
Director → TechLead → specialist engineers (algorithm / memory / compute / host_runtime), multi-round and
budget-controlled, with each patch independently verified before it's accepted.

The workflow for this task:

1. **Baseline profiling** — benchmark the naive kernel, identify bottlenecks.
2. **Optimization** — engineers produce optimized kernels in parallel (`round_1/engineer_0`, `engineer_1`).
3. **Verification** — compile, correctness check, and performance benchmark against baseline.
4. **Selection** — pick the best-performing kernel by unweighted geomean speedup.

---

2. Baseline Kernel

The naive FP32 GEMM kernel in `src/gemm_cuda.hip` computes `C = A @ B` (standard) or `C = A @ B.T` (transpose_b). One thread per output element, no data reuse, no shared memory.

```
__global__ void gemm_kernel(int M, int N, int K, int transpose_b,
                            const float *__restrict__ A,
                            const float *__restrict__ B,
                            float *__restrict__ C) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int total = M * N;
    if (idx >= total) return;

    int row = idx / N;
    int col = idx % N;

    float acc = 0.f;
```

```
    const float *a_row = A + row * K;
    if (transpose_b) {
        // B is (N, K): element (k, col) lives at B[col * K + k]
        for (int k = 0; k < K; ++k) {
            acc += a_row[k] * B[col * K + k];
        }
    } else {
        // B is (K, N): element (k, col) lives at B[k * N + col]
        for (int k = 0; k < K; ++k) {
            acc += a_row[k] * B[k * N + col];
        }
    }
    C[row * N + col] = acc;
}
```

Why it is slow:

- **Zero data reuse** — every thread independently fetches its own K-length row of A and K-length column of B from global memory. With M×N threads, that is O(M×N×K) total bytes moved and an arithmetic intensity of only ~0.25 FLOP/byte — far below the roofline.
- **Uncoalesced B loads (dominant bottleneck)** — in `transpose_b` and `a_at`, thread `col` reads `B[col*K + k]`. Since consecutive threads have consecutive `col` values but K elements apart, a single 64-byte cache line serves just 1 useful float per thread. This 16x waste in DRAM traffic is why `transpose_b` takes 16× longer than `standard` at the same shape (1.341 ms vs 0.083 ms for shape_4).
- **Runtime branch** — `transpose_b` is a runtime `int` checked every iteration of the K-loop, preventing the compiler from specializing either path.

---

3. How to Run

```
use path_to_GEAK/kernel_workflow to optimize
path_to_GEAK/examples/tasks/knn
```

Configuration

- **GPU**: AMD Instinct MI300X VF (gfx942, CDNA3, 304 CUs), ROCm 6+.
- **Layouts**: `standard` (B is K x N), `transpose_b` (B is N x K), `a_at` (A x A.T, output is M x M).
- **Correctness**: `atol = rtol = 1e-3`, comparing against CPU `torch.mm` reference.
- **Benchmark settings**: 10 warmup + 50 timed iterations per case, CUDA event timing.

---

4. Optimized Kernel

Engineer 0 produced the winning kernel — an LDS-tiled, register-blocked GEMM with compile-time layout dispatch.

Key changes (full diff: gemm/final_patch_gemm_cuda_hip.diff)

4.1 LDS Tiling with Coalesced Loads

Problem it solves: Zero data reuse and uncoalesced B loads .

In the baseline, every thread fetches its own K-length row of A and column of B directly from global memory — no thread reuses another thread's data, and the `transpose_b` access pattern is stride-K (uncoalesced).

The fix: each thread block cooperatively loads a BM×BK slab of A and a BK×BN slab of B into `__shared__` memory (LDS) once per K-step. All 256 threads in the block participate in the load using `float4` vectorized loads, so every global memory transaction is fully coalesced regardless of A or B's layout. The data is then reused by all threads in the block as they compute their output tile — eliminating both the redundant global traffic and the uncoalesced penalty.

For `transpose_b` specifically, what was `B[col*K + k]` (stride-K, 1 useful float per cache line) becomes a cooperative `float4` load along the contiguous K dimension, making it fully coalesced.

```
__shared__ float As[BK][BM];
__shared__ float Bs[BK][BN];
// ...
if constexpr (TRANSPOSED) {
    float4 bv = *reinterpret_cast<const float4 *>(
        &B[(colStart + bN) * K + k0 + bK4 * 4]);
    Bs[bK4 * 4 + 0][bN] = bv.x;
    // ...
} else {
    float4 bv = *reinterpret_cast<const float4 *>(
        &B[(k0 + bRowS) * N + colStart + bCol4S * 4]);
    *reinterpret_cast<float4 *>(&Bs[bRowS][bCol4S * 4]) = bv;
}
```

4.2 Register Micro-Tiling

Problem it solves: Zero data reuse — even after LDS tiling, if each thread only computes one output element, the LDS data is loaded but not reused enough within each thread.

Each thread now computes a 4×4 micro-tile of C (16 output elements) instead of 1. Within each K-step, the thread loads a 4-element column of A and a 4-element row of B from LDS

into registers, then accumulates all 16 outer-product combinations. This means each LDS load is reused 16× within a single thread, and across the 256 threads in the block the total LDS bandwidth is well-utilized. The `#pragma unroll` directives ensure the compiler fully unrolls the inner loops for maximum instruction-level parallelism.

```
float acc[TM][TN];
// ...
#pragma unroll
for (int k = 0; k < BK; ++k) {
    float regA[TM];
    float regB[TN];
    // load 4-element slices from LDS into registers (float4 vectorized)
    for (int i = 0; i < TM; i += 4)
        *reinterpret_cast<float4 *>(&regA[i]) =
            *reinterpret_cast<const float4 *>(&As[k][threadRow * TM + i]);
    for (int j = 0; j < TN; j += 4)
        *reinterpret_cast<float4 *>(&regB[j]) =
            *reinterpret_cast<const float4 *>(&Bs[k][threadCol * TN + j]);
    // outer-product accumulation: 4x4 = 16 FMAs per k-step
    for (int i = 0; i < TM; ++i)
        for (int j = 0; j < TN; ++j)
            acc[i][j] = fmaf(regA[i], regB[j], acc[i][j]);
}
```

4.3 Compile-Time Layout Dispatch

Problem it solves: Runtime branch in the hot loop .

The baseline checked `if (transpose_b)` on every iteration of the K-loop. Since `transpose_b` is the same value for every iteration, this is redundant work and prevents the compiler from optimizing either branch. The fix: `transpose_b` is now a `template<bool TRANSPOSED>` parameter. The launcher checks the flag once and dispatches to the correct template instantiation, so inside the kernel the layout is known at compile time. The compiler can then fully optimize each path independently, and `if constexpr` eliminates the dead branch entirely.

```
template <bool TRANSPOSED, int BM, int BN, int BK, int TM, int TN>
__global__ void __launch_bounds__((BM / TM) * (BN / TN))
gemm_kernel_tiled(int M, int N, int K, const float *A, const float *B,
float *C);

template <int BM, int BN, int BK, int TM, int TN>
static void launch_tiled(int M, int N, int K, int transpose_b, ...) {
    if (transpose_b) {
        gemm_kernel_tiled<true, BM, BN, BK, TM, TN><<<blocks, threads>>>
(M, N, K, A, B, C);
    } else {
```

```
        gemm_kernel_tiled<false, BM, BN, BK, TM, TN><<<blocks, threads>>>
(M, N, K, A, B, C);
    }
}
```

4.4 Bounds-Checked Fallback

The tiled kernel assumes M, N, K are multiples of the tile dimensions (BM=64, BN=64, BK=16). Shapes that don't satisfy this (e.g. 37×17×53, 129×31×257) would produce incorrect results without a fallback. A separate scalar code path handles these ragged dimensions by checking bounds on every global memory access and zero-padding out-of-bounds reads. This ensures correctness for arbitrary M, K, N without slowing down the fast path taken by all aligned benchmark shapes.

4.5 Tile Configuration

Parameter	Value	Rationale
BM, BN	64	64×64 output tile per block — large enough for data reuse, small enough to saturate 304 CUs
BK	16	K-dimension tile depth — balances LDS usage vs. reuse depth
TM, TN	4	4×4 micro-tile per thread — each thread computes 16 outputs, maximizing register reuse
Threads/block	256	(64/4) × (64/4) — one thread per micro-tile position
Grid	ceil(N/BN) × ceil(M/BM)	2D grid over output tiles — covers the full M×N output

---

5. Output Files

File	Description
baseline_timing.json	Baseline per-case latencies (15 cases)
baseline_metrics.json	Baseline profiling summary and bottleneck analysis
final_patch_gemm_cuda_hip.diff	Clean unified diff of the optimized kernel
round_1/engineer_0/workspace/src/gemm_cuda.hip	Optimized kernel source
round_1/engineer_0/workspace/build/performance_report.json	Optimized kernel benchmark results (15

File	Description
	cases)
round_1/engineer_0/verify/	Verification workspace with full benchmark run
gemm_report_20260715_165107.json	Full benchmark report (39 correctness + 33 perf cases)
profiling_summary.md	Detailed baseline profiling analysis

---

6. Results and Conclusion

6.1 Performance Comparison

Shape (M,K,N)	Layout	Baseline (ms)	Optimized (ms)	Speedup
1024,256,1024	transpose_b	1.342	0.021	**65x**
1024,256,1024	a_at	1.330	0.021	**63x**
512,512,512	transpose_b	0.723	0.031	**24x**
512,512,512	a_at	0.718	0.031	**23x**
256,512,256	transpose_b	0.173	0.030	**5.7x**
1024,256,1024	standard	0.083	0.021	**4.0x**
128,256,128	transpose_b	0.082	0.016	**5.2x**
64,64,64	all	~0.013	~0.014	~1x (launch-floor)

Geomean speedup: 5.24x

The largest wins are on `transpose_b` and `a_at` layouts, whose uncoalesced `B[col*K+k]` reads dominated the baseline. The tiled kernel stages both operands into LDS with fully coalesced `float4` loads regardless of layout.

Small shapes (64^3) sit at the ~0.014 ms CUDA-event launch floor and show no real kernel improvement — this is expected.

6.2 Correctness

All **39/39 correctness cases pass** (11 shapes + 2 ragged shapes x 3 layouts), including ragged shapes (37x17x53, 129x31x257) that exercise the bounds-checked fallback, and large matmuls up to 4096x6144x12288.